State Estimation and Filtering for Robotic Perception

AA273

Unscented Kalman Filter, Particle Filter



Stanford University

MSL

AA273: State Estimation and Filtering for Robotic Perception

Announcements:

- HW 4 due Fri Feb 13
- HW 5 out Fri Feb 13, due Fri Feb 20
- Group signup sheet posted (groups from 1 – 4 people)
- Project Proposal: due Tue Feb 17
  - 2-3 pages, Latex
  - Related work >= 10 refs, ~1 page
  - Proposed work ~ 1 page
  - Bibtex refs

Lecture 11:

- Unscented Kalman filter
- Proof of unscented transform invertibility
- Importance sampling
- Particle filter

Recall: Nonlinear SS Model and EKF

Dynamics eqn.:

$$X_{t+1} = f(X_t, u_t) + W_t$$

Measurement eqn.:

$$Y_t = g(X_t) + V_t$$

Uncorrelated IC, and noise processes:

$X_0 \sim \mathcal{N}(\mu_{0|0}, \Sigma_{0|0})$ $\quad X_0, W_t, V_\tau$

$W_t \sim GWN(0, Q_t)$ $\quad$ independent

$V_t \sim GWN(0, R_t)$ $\quad$ for all t, τ

Jacobian matrices A_t and C_t

$$A_t = \nabla_{x_t} f(\mu_{t|t}, u_t)$$

$$C_t = \nabla_{x_t} g(\mu_{t|t-1})$$

Given: $\mu_{0|0}, \Sigma_{0|0}, f(\cdot,\cdot), g(\cdot), Q_t, R_t$

For: $t = 1, 2, \ldots$

Prediction:

$$\mu_{t|t-1} = f(\mu_{t-1|t-1}, u_{t-1})$$

$$\Sigma_{t|t-1} = A_{t-1}\Sigma_{t-1|t-1}A_{t-1}^T + Q_{t-1}$$

Update:

$$\mu_{t|t} = \mu_{t|t-1} + K_t(y_t - g(\mu_{t|t-1}))$$

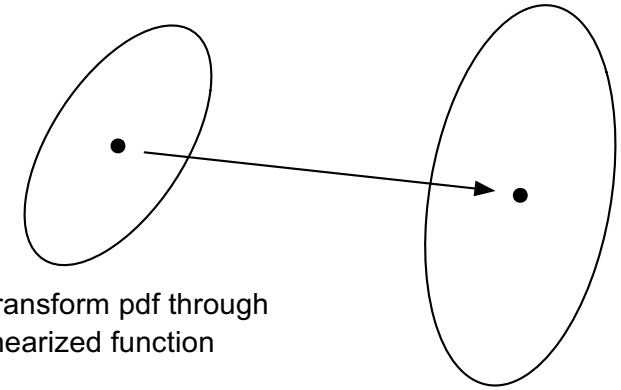
$$\Sigma_{t|t} = \Sigma_{t|t-1} - K_t C_t \Sigma_{t|t-1}$$

$$K_t = \Sigma_{t|t-1}C_t^T(C_t\Sigma_{t|t-1}C_t^T + R_t)^{-1}$$

Loop

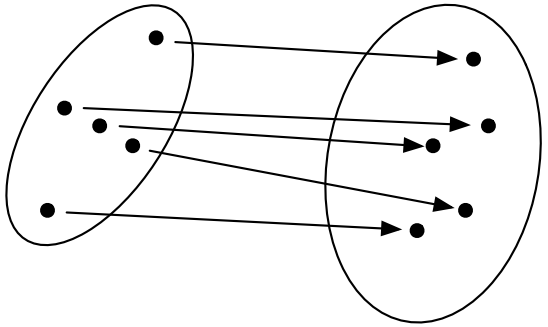
Recall: Unscented Kalman filter (UKF)

EKF



Transform pdf through
linearized function

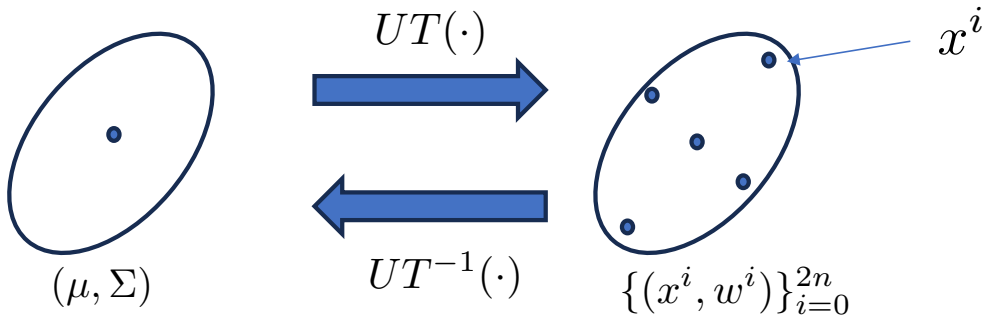
UKF



1. Compute sigma-points
2. Transform each sigma point through nonlinear function
3. Compute Gaussian from the transformed and weighted sigma-points

Unscented Transform
(a.k.a sigma-point transform)

$$UT(\mu, \Sigma) = \begin{cases} x^0 = \mu, & w^0 = \frac{\lambda}{\lambda+n} \\ x^i = \mu + (\sqrt{(\lambda+n)\Sigma})_i, & w^i = \frac{1}{2(\lambda+n)} \quad i = 1, \ldots, n \\ x^{n+i} = \mu - (\sqrt{(\lambda+n)\Sigma})_i, & w^i = \frac{1}{2(\lambda+n)} \quad i = 1, \ldots, n \end{cases}$$



$$UT^{-1}(\{(x^i, w^i)\}_{i=0}^{2}n) = \begin{cases} \mu = \sum_{i=0}^{2n} w^i x^i \\ \Sigma = \sum_{i=0}^{2n} w^i (x^i - \mu)(x^i - \mu)^T \end{cases}$$

Unscented Transform Proof of Invertibility

Mean:

$$\mu = \sum_{i=0}^{2n} w^i x^i$$

On the board, in the notes.

Cov:

$$\Sigma = \sum_{i=0}^{2n} w^i (x^i - \mu)(x^i - \mu)^T$$

Note: $$MM^T = [m_1 \quad \cdots \quad m_n] \begin{bmatrix} m_1^T \\ \vdots \\ m_n^T \end{bmatrix} = \sum_{i=1}^{n} m_i m_i^T$$

UKF Algorithm

Given: $\mu_{0|0},\ \Sigma_{0|0},\ f(\cdot,\cdot),\ g(\cdot),\ Q_t,\ R_t$

For: $t = 1, 2, \ldots$

Predict:

$$UT(\mu_{t-1|t-1}, \Sigma_{t-1|t-1}) \rightarrow \{(x^i_{t-1|t-1}, w^i)\}_{i=0}^{2n}$$

Predicted Sig-points $\overline{x}^i_{t|t-1} = f(x^i_{t-1|t-1}, u_{t-1})$

w^i no change

Inv UT plus noise $\mu_{t|t-1} = \sum_{i=0}^{2n} w^i \overline{x}^i_{t|t-1}$

Add process noise Cov

$$\Sigma_{t|t-1} = \sum_{i=0}^{2n} w^i (\overline{x}^i_{t|t-1} - \mu_{t|t-1})(\overline{x}^i_{t|t-1} - \mu_{t|t-1})^T + Q_{t-1}$$

Prediction Mean, Cov $(\mu_{t|t-1}, \Sigma_{t|t-1})$

Update:

$$UT(\mu_{t|t-1}, \Sigma_{t|t-1}) \rightarrow \{(x^i_{t|t-1}, w^i)\}_{i=0}^{2n}$$

Predicted Sig-measurements $y^i_{t|t-1} = g(x^i_{t|t-1})$

Predicted mean measurement $\hat{y}_{t|t-1} = \sum_{i=0}^{2n} w^i y^i_{t|t-1}$

Add measure noise Cov

Meas. Cov $\Sigma_y = \sum_{i=0}^{2n} w^i (y^i_{t|t-1} - \hat{y}_{t|t-1})(y^i_{t|t-1} - \hat{y}_{t|t-1})^T + R_t$

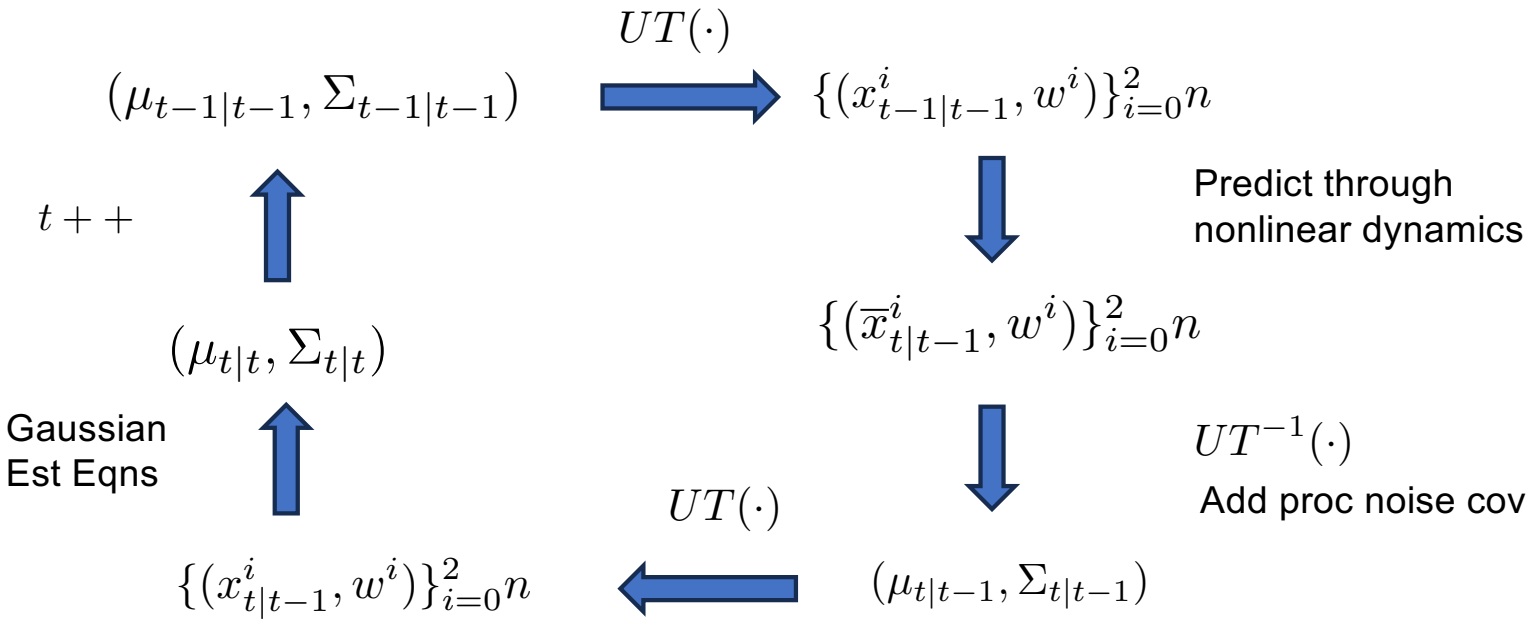
Cross Cov $\Sigma_{xy} = \sum_{i=0}^{2n} w^i (x^i_{t|t-1} - \mu_{t|t-1})(y^i_{t|t-1} - \hat{y}_{t|t-1})^T$

Gaussian Est Eqns

$$\mu_{t|t} = \mu_{t|t-1} + \Sigma_{xy}\Sigma_y^{-1}(y_t - \hat{y}_{t|t-1})$$
$$\Sigma_{t|t} = \Sigma_{t|t-1} - \Sigma_{xy}\Sigma_y^{-1}\Sigma_{xy}^T$$

Posterior Mean, Cov $(\mu_{t|t}, \Sigma_{t|t})$

UKF Algorithm Visualization



UKF Algorithm Discussion

- No Jacobians required!
- Dynamics and measurement can be complex simulations or learned models.
- Highly parallelizable for GPUs
- Requires about 3x more computation time per step than EKF, usually negligible

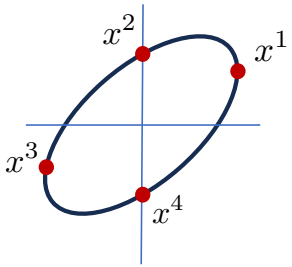
Matrix Square Roots

Recall definition: $\{\sqrt{\Sigma} \mid \sqrt{\Sigma}\sqrt{\Sigma}^T = \Sigma\}$

Multiple choices:

(a) Cholesky sqrt: $\Sigma = LL^T$

Lower triangular gives: $\left(\sqrt{\Sigma}\right)_j = \begin{bmatrix} 0 \\ \vdots \\ 0 \\ l_{ij} \\ \vdots \\ l_{in} \end{bmatrix}$



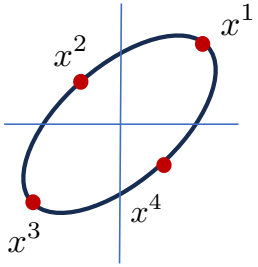
(b) ellipse-aligned sqrt: $\Sigma = V\Lambda V^T$

eigvects

diag, eigvals

Define: $\sqrt{\Sigma} := V\sqrt{\Lambda}$

$\sqrt{\Sigma}\sqrt{\Sigma}^T := V\sqrt{\Lambda}\sqrt{\Lambda}^T V^T = \Sigma$

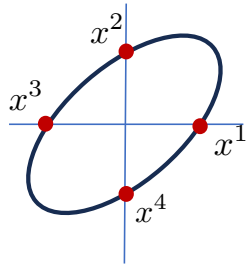


(b) axis-aligned sqrt:

Define: $\sqrt{\Sigma} := V\sqrt{\Lambda}V^T$

$\sqrt{\Sigma}\sqrt{\Sigma}^T := (V\sqrt{\Lambda}V^T)(V\sqrt{\Lambda}^T V^T) = \Sigma$

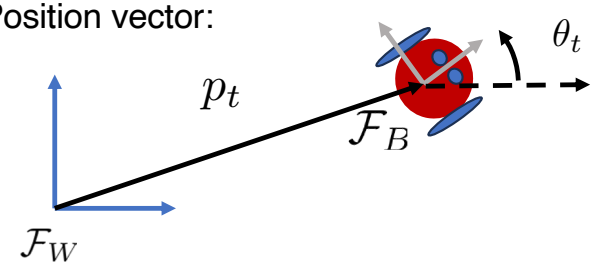
V orthogonal matrix: $V^T V = I$



Example: Differential Drive Robot

Heading angle:

Position vector:



Robot state:

$$x_t = \begin{bmatrix} p_t^x \\ p_t^y \\ \theta_t \end{bmatrix}$$

Control input:

$$u_t = \begin{bmatrix} v_t \\ \omega_t \end{bmatrix}$$

Robot dynamics model cont. time:

$$\begin{aligned} \dot{p}_t^x &= v_t \cos\theta_t \\ \dot{p}_t^y &= v_t \sin\theta_t \\ \dot{\theta}_t &= \omega_t \end{aligned}$$

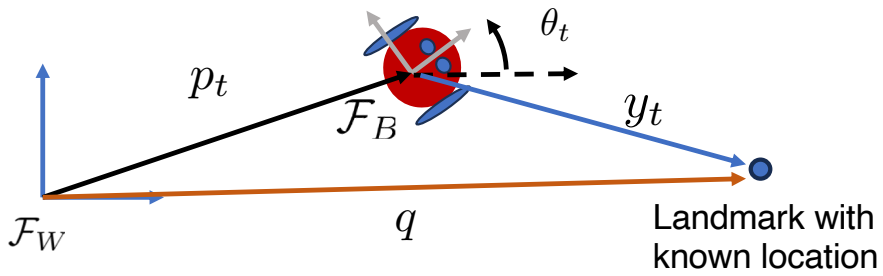
Robot dynamics model discrete. time with noise:

$$\begin{aligned} p_{t+1}^x &= p_t^x + \delta_t v_t \cos\theta_t + w_t^x \\ p_{t+1}^y &= p_t^y + \delta_t v_t \sin\theta_t + w_t^y \\ \theta_{t+1} &= \theta_t + \delta_t \omega_t + w_t^\theta \end{aligned}$$

UKF prediction:

$$\overline{x}_{t|t-1}^i = f(x_{t-1|t-1}^i, u_{t-1})$$

Example: Range Sensor Model



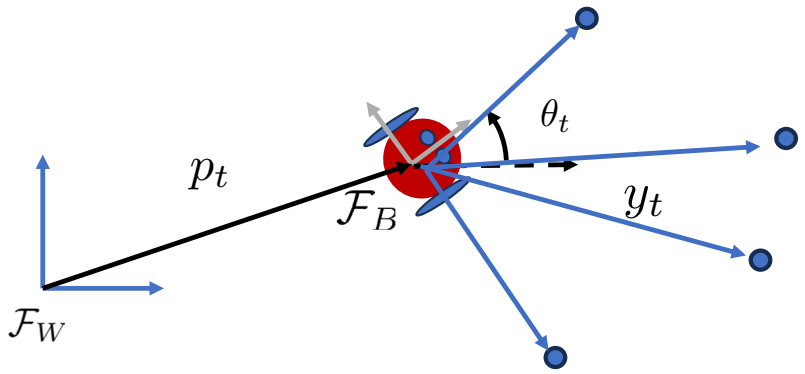
Range sensing model:

$$y_t = \|p_t - q\| + v_t$$

UKF predicted measurement:

$$y^i_{t|t-1} = g(x^i_{t|t-1})$$

Examples: Range Sensor Model



With multiple landmarks, just stack measurements:

$$y_t = \begin{bmatrix} y_t^1 \\ \vdots \\ y_t^N \end{bmatrix}$$

Range sensing model for each landmark i:

$$y_t^i = \|p_t - q_i\| + v_t^i$$

UKF predicted measurement for particle i measurement j:

$$y_{t|t-1}^{ij} = \|p_{t|t-1}^i - q_j\|$$

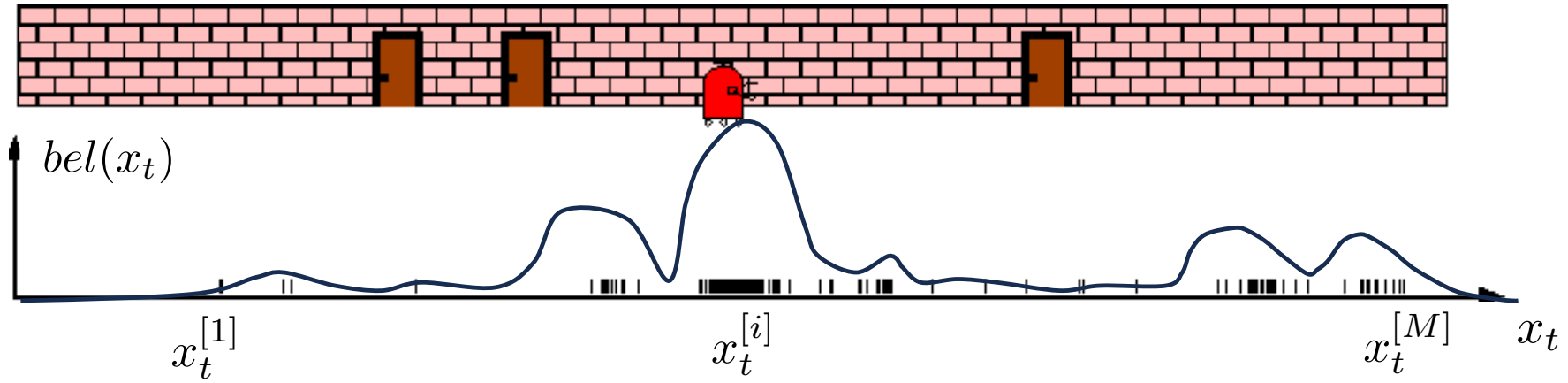
UKF stacked predicted measurement:

$$y_{t|t-1}^i = \begin{bmatrix} y_{t|t-1}^{i1} \\ \vdots \\ y_{t|t-1}^{iN} \end{bmatrix}$$

This is called UKF localization.

Particle filter

- Key idea: represent posterior $bel(x_t)$ by a set of random samples
- Can represent arbitrary distributions
- Requires much more computation than KF/EKF



- Allows one to represent non-Gaussian distributions and handle nonlinear transformations in a direct way

Particle filter

- Samples of posterior distribution are called *particles*, denoted as

$$\mathcal{X}_t := x_t^{[1]}, x_t^{[2]}, \ldots, x_t^{[M]}$$

- A particle represents a hypothesis about what the true world state might be at time *t*
- Ideally, particles should be distributed according to

$$x_t^{[m]} \sim p(x_t \mid y_{1:t}) = bel(x_t)$$

- Matching exact only as $M \to \infty$, but $M \approx 10^n - 100^n$ usually good enough for n dimensional state space
- A particle filter constructs the particle set $\mathcal{X}_t$ from the particle set $\mathcal{X}_{t-1}$ recursively, with the goal of matching the distribution $bel(x_t)$

Particle filter: algorithm

- The temporary particle set $\overline{\mathcal{X}}_t$ represents the belief $\overline{bel}(x_t)$
- The particle set $\mathcal{X}_t$ represents the belief $bel(x_t)$
- Importance factors are used to incorporate measurement z_t in the particle set
- After resampling, particles are (as $M \to \infty$) distributed as

$$bel(x_t) = \eta\, p(z_t \mid x_t^{[m]})\, \overline{bel}(x_t)$$

$bel(x_{t-1})$

Data: $\mathcal{X}_{t-1}, u_t, z_t$
Result: $\mathcal{X}_t$
$\overline{\mathcal{X}}_t = \mathcal{X}_t = \emptyset;$
for $m = 1$ **to** M **do**
  Sample $x_t^{[m]} \sim p(x_t \mid u_t, x_{t-1}^{[m]});$
  $w_t^{[m]} = p(z_t \mid x_t^{[m]});$
  $\overline{\mathcal{X}}_t = \overline{\mathcal{X}}_t \cup \left(x_t^{[m]}, w_t^{[m]}\right);$
end
for $m = 1$ **to** M **do**
  Draw i with probability $\propto w_t^{[i]};$
  Add $x_t^{[i]}$ to $\mathcal{X}_t;$
end
Return $\mathcal{X}_t$

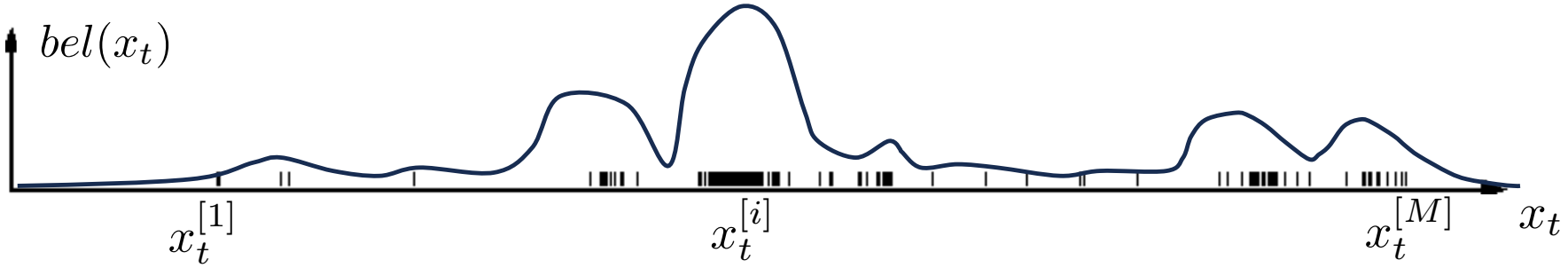
$bel(x_t)$

Prediction: $\overline{bel}(x_t)$

Importance factor

Resample: $bel(x_t)$

Representing distributions as particle sets



The particles let us **approximate** any desired probability or moment:

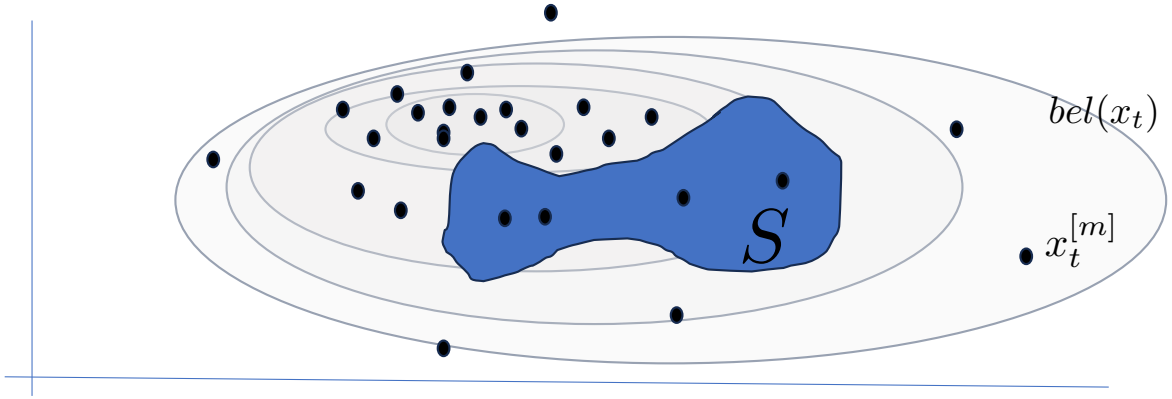
Mean: $\mu_t \approx \frac{1}{M} \sum_{m=1}^{M} x_t^{[m]} = \tilde{\mu}_t$

Covariance: $\Sigma_t \approx \frac{1}{M} \sum_{m=1}^{M} (x_t^{[m]} - \tilde{\mu}_t)(x_t^{[m]} - \tilde{\mu}_t)^T = \tilde{\Sigma}_t$

Probability:

$$Pr(x_t \in S) \approx \frac{\#\text{ particles in S}}{M} = \frac{|\{m \mid x_t^{[m]} \in S\}|}{M}$$

Representing distributions as particle sets



The particles let us **approximate** any desired probability or moment:

Mean: $\mu_t \approx \frac{1}{M} \sum_{m=1}^{M} x_t^{[m]} = \tilde{\mu}_t$

Covariance: $\Sigma_t \approx \frac{1}{M} \sum_{m=1}^{M} (x_t^{[m]} - \tilde{\mu}_t)(x_t^{[m]} - \tilde{\mu}_t)^T = \tilde{\Sigma}_t$

Probability:

$$Pr(x_t \in S) \approx \frac{\#\text{ particles in S}}{M} = \frac{|\{m \mid x_t^{[m]} \in S\}|}{M}$$

Empirical vs Integral Expectations

Integral Expectation:

$$\mu_t = E[X_t] = \int_{x_t} x_t p(x_t)\, dx_t$$

$$\Sigma_t = Cov(X_t) = \int_{x_t} (x_t - \mu_t)(x_t - \mu_t)^T p(x_t)\, dx_t$$

$$Pr(x_t \in S) = \int_{x_t \in S} p(x_t)\, dx_t$$

$$= \int_{x_t} \mathbf{1}_S(x_t) p(x_t)\, dx_t = E[1_S(x_t)]$$

$$E[f(X_t)] = \int_{x_t} f(x_t) p(x_t)\, dx_t$$

Empirical (a.k.a Monte Carlo estimate):

$$\mu_t \approx \frac{1}{M} \sum_{m=1}^{M} x_t^{[m]} = \tilde{\mu}_t$$

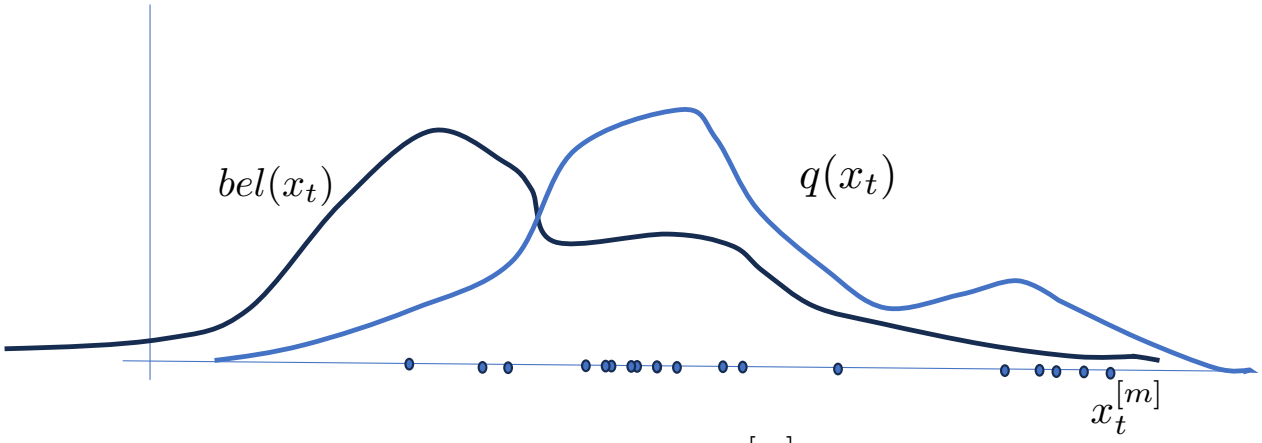
$$\Sigma_t \approx \frac{1}{M} \sum_{m=1}^{M} (x_t^{[m]} - \tilde{\mu}_t)(x_t^{[m]} - \tilde{\mu}_t)^T = \tilde{\Sigma}_t$$

$$Pr(x_t \in S) \approx \frac{\#\text{ particles in S}}{M} = \frac{|\{m \mid x_t^{[m]} \in S\}|}{M}$$

$$E[f(X)] \approx \frac{1}{M} \sum_{m=1}^{M} f(x_t^{[m]})$$

Law of Large Numbers: $\lim_{M \to \infty} \left(\frac{1}{M} \sum_{i=1}^{N} f(X_i) \right) = E[f(X)]$ almost surely

Importance Sampling



We want to represent $bel(x_t)$ with particles $x_t^{[m]} \sim q(x_t)$ from a different distribution

Weight the particles!

$$E_{bel}[f(x_t)] = \int_{x_t} f(x_t) bel(x_t)\, dx_t = \int_{x_t} f(x_t) \frac{bel(x_t)}{q(x_t)} q(x_t)\, dx_t$$

We have:

$$w(x) = \frac{bel(x)}{q(x)}$$

$$E_{bel}[f(x_t)] = \int_{x_t} f(x_t) w(x_t) q(x_t)\, dx_t \approx \sum_{x_t^{[m]}} f(x_t^{[m]}) w_t^{[m]}$$

Importance weights:

$$w_t^{[m]} = \frac{bel(x_t^{[m]})}{M q(x_t^{[m]})}$$

Stochastic Measurement Model

Measurement distribution:

$$p(z_t \mid x_t)$$

We **evaluate probabilities** from this model to get weights.

$$w_t^{[m]} = p(z_t \mid x_t^{[m]})$$

Nonlinear state space form:

$$z_t = h(x_t) + \delta_t$$

$$\delta_t \sim \mathcal{D}(\delta; \theta)$$ Non-Gaussian or Gaussian measurement noise

Plug $z_t, x_t^{[m]}$ into formula for $\mathcal{D}(z_t - h(x_t^{[m]}); \theta)$ to obtain $w_t^{[m]}$

Stochastic Measurement Model

E.g., Gaussian measurement noise

$$\delta_t \sim \mathcal{N}(\delta; 0, Q_t) = \frac{1}{\sqrt{(2\pi)^n |Q_t|}} e^{-\frac{1}{2}\delta^T Q_t^{-1} \delta}$$

Plug $z_t, x_t^{[m]}$ into

$$w_t^{[m]} = \mathcal{N}(z_t - h(x_t^{[m]}); 0, Q_t) = \frac{1}{\sqrt{(2\pi)^n |Q_t|}} e^{-\frac{1}{2}(z_t - h(x_t^{[m]}))^T Q_t^{-1} (z_t - h(x_t^{[m]}))}$$

To get $w_t^{[m]}$

Stochastic Dynamics Model

State transition distribution:

$$p(x_t \mid x_{t-1}, u_{t-1})$$

We **draw samples** from this model to get new particles at time *t*.

$$x_t^{[m]} \sim p(x_t \mid x_{t-1}^{[m]}; u_{t-1})$$

Nonlinear state space model:

$$x_t = f(x_{t-1}, u_{t-1}) + w_{t-1}$$

$$w_t \sim \mathcal{D}(w_t)$$ Non-Gaussian or Gaussian process noise

(1) Draw noise sample $w_{t-1}^{[m]}$

(2) Plug into nonlinear state space model to get $x_t^{[m]} = f(x_{t-1}^{[m]}, u_{t-1}) + w_{t-1}^{[m]}$

This draws $x_t^{[m]}$ from the state transition distribution $x_t^{[m]} \sim \mathcal{D}(x_t - f(x_{t-1}^{[m]}, u_{t-1}))$

Stochastic Dynamics Model

E.g., Gaussian process noise

(1) Draw: $w_{t-1}^{[m]} \sim \mathcal{N}(0, R_{t-1})$

(2) Evaluate: $x_t^{[m]} = f(x_{t-1}^{[m]}, u_{t-1}) + w_{t-1}^{[m]}$

Equivalent to drawing from non-gaussian state transition distribution:

$$x_t^{[m]} \sim \mathcal{N}(x_t - f(x_{t-1}^{[m]}, u_{t-1}); 0, R_{t-1})$$

Particle filter: algorithm

- The temporary particle set $\overline{\mathcal{X}}_t$ represents the belief $\overline{bel}(x_t)$
- The particle set $\mathcal{X}_t$ represents the belief $bel(x_t)$
- Importance factors are used to incorporate measurement z_t in the particle set
- After resampling, particles are (as $M \to \infty$) distributed as

$$bel(x_t) = \eta\, p(z_t \mid x_t^{[m]})\, \overline{bel}(x_t)$$

$bel(x_{t-1})$

Data: $\mathcal{X}_{t-1}, u_t, z_t$
Result: $\mathcal{X}_t$
$\overline{\mathcal{X}}_t = \mathcal{X}_t = \emptyset;$
for $m = 1$ **to** M **do**
  Sample $x_t^{[m]} \sim p(x_t \mid u_t, x_{t-1}^{[m]});$
  $w_t^{[m]} = p(z_t \mid x_t^{[m]});$
  $\overline{\mathcal{X}}_t = \overline{\mathcal{X}}_t \cup \left(x_t^{[m]}, w_t^{[m]}\right);$
end
for $m = 1$ **to** M **do**
  Draw i with probability $\propto w_t^{[i]};$
  Add $x_t^{[i]}$ to $\mathcal{X}_t;$
end
Return $\mathcal{X}_t$

$bel(x_t)$

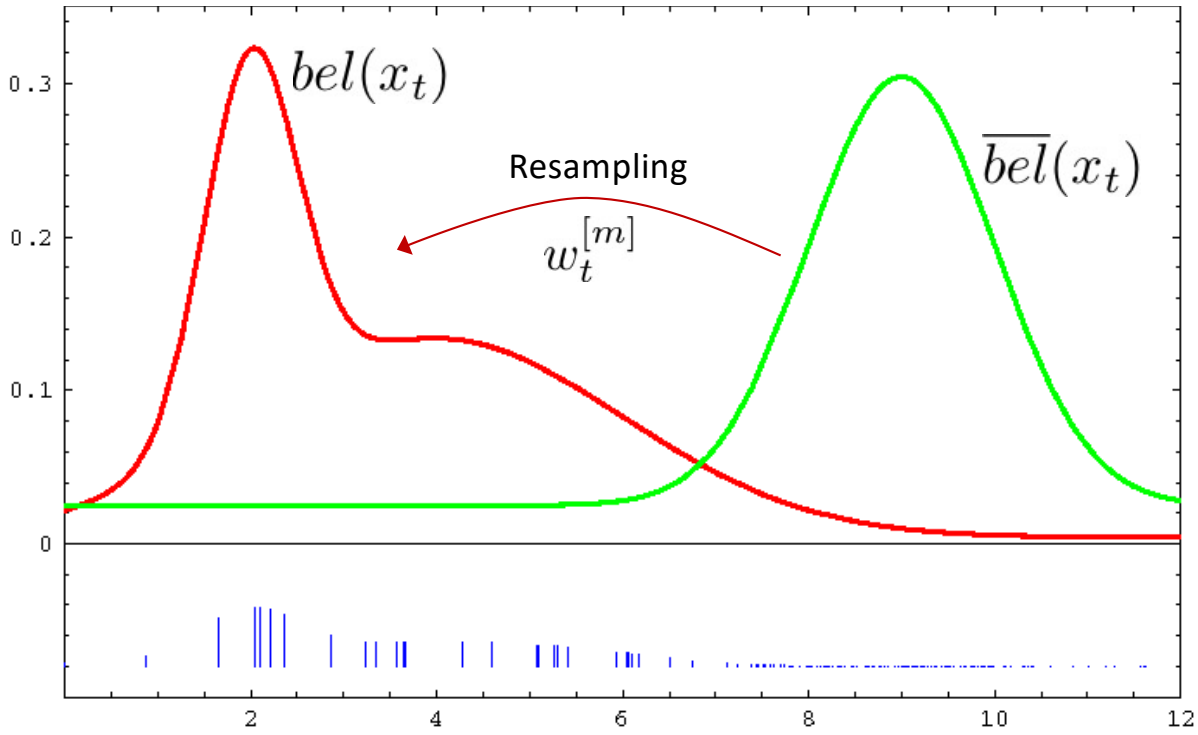
Prediction: $\overline{bel}(x_t)$

Importance factor

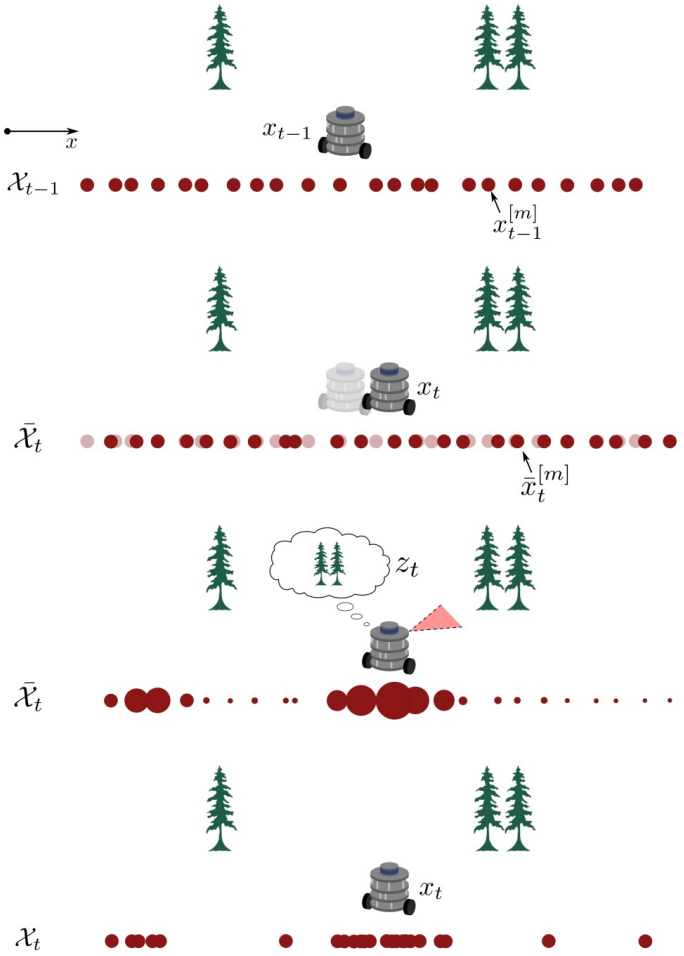
Resample: $bel(x_t)$

Resampling

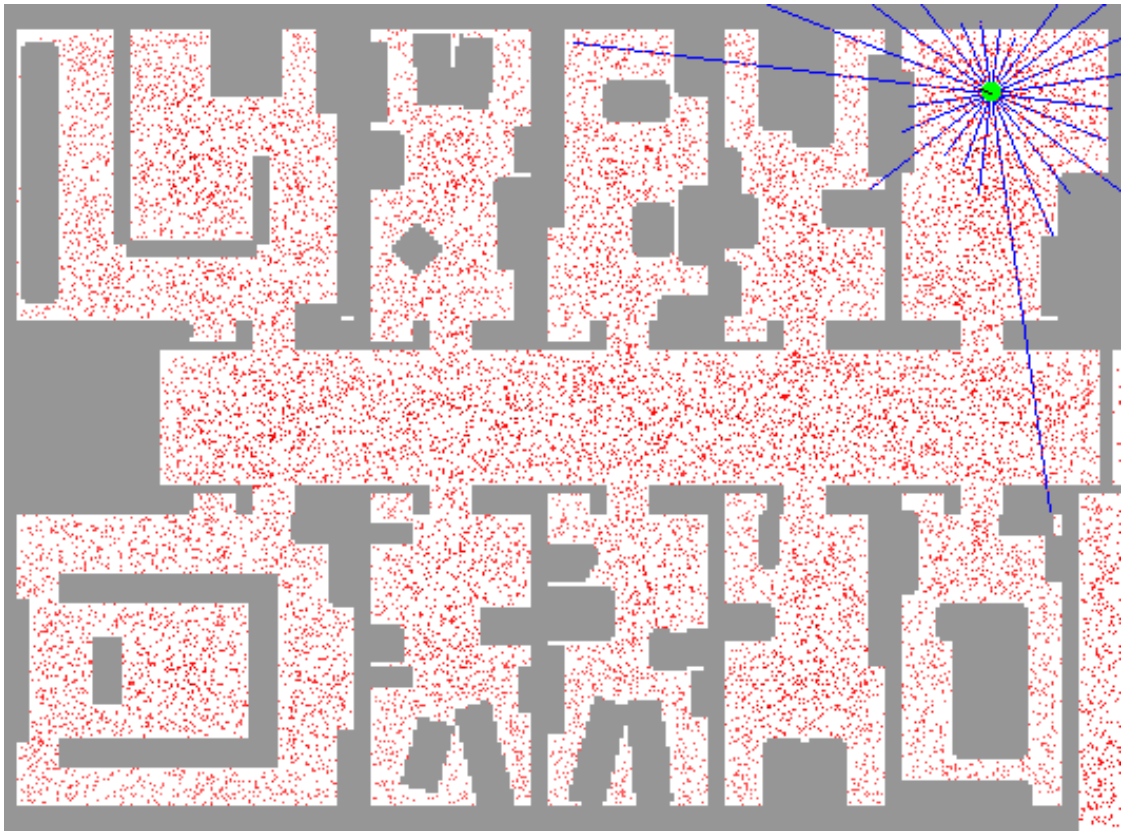
- Variants exist to sample less frequently (not every time step) to avoid particle diversity problems.



Particle filter: example

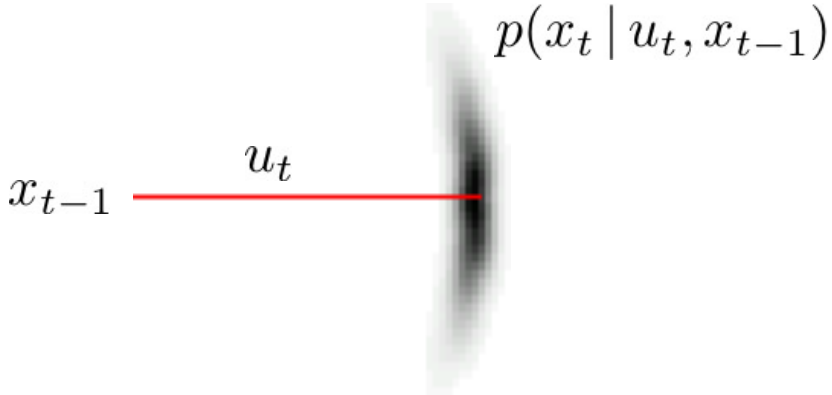


MCL: example



E.g.: Monte Carlo localization

- Motion model is probabilistic



PF requires us to sample:

$$x_t^{[m]} \sim p(x_t \mid u_t, x_{t-1}^{[m]})$$

Example for dynamics with additive noise:

$$x_t^{[m]} = f(x_{t-1}^{[m]}, u_t) + \epsilon_{t-1}^{[m]}, \quad \epsilon_{t-1}^{[m]} \sim \mathcal{N}(0, R_{t-1})$$

Also requires to evaluate measurement likelihood:

$$w_t^{[m]} = p(z_t \mid x_t^{[m]})$$

E.g., for Gaussian:

$$w_t^{[m]} = \mathcal{N}(C_t x_t^{[m]}, Q_t)$$

Monte Carlo localization

- Key idea: represent belief $bel(x_t)$ by a set of *M* particles

$$\mathcal{X}_t = \{x_t^{[1]}, x_t^{[2]}, \ldots, x_t^{[M]}\}$$

- Requires a map *m* as input
- Can handle dynamic environments via outlier rejection

Compute for occupancy map
Or feature based map

Data: $\mathcal{X}_{t-1}, u_t, z_t, m$
Result: $\mathcal{X}_t$
$\overline{\mathcal{X}}_t = \mathcal{X}_t = \emptyset$;
for $i = 1$ **to** M **do**
  Sample $x_t^{[m]} \sim p(x_t \mid u_t, x_{t-1}^{[m]}, m)$;
  $w_t^{[m]} = p(z_t \mid x_t^{[m]}, m)$;
  $\overline{\mathcal{X}}_t = \overline{\mathcal{X}}_t \cup \left(x_t^{[m]}, w_t^{[m]}\right)$;
end
for $i = 1$ **to** M **do**
  Draw i with probability $\propto w_t^{[i]}$;
  Add $x_t^{[i]}$ to $\mathcal{X}_t$;
end
Return $\mathcal{X}_t$

Sample reject collisions